GIU Berlin

Mechatronics Engineering (MCTR B501)

Smart train based Railway Gate

Important Note: If this is your proposal, don't populate all sections but stick to the info in slide 4 in the Project Description.

Name	ID
Youssef hossam	
Ahmed wassem	
Moamen hany	

Table of Contents

1. Project Description 3
2. Methodology 7
2.1 Mechanical design 7
2.2 Electrical design 8
2.3 Control 9
2.3.1. Modeling 9
2.3.2. Analysis 9
2.3.3. Controller Design 9
2.4 Programming 11
3. Design Evaluation 25
4. Appendix 26

1. Project Description

This project aims to develop an automated and manual train crossing system using the STM32F103C8T6 microcontroller. The system controls a toy train's motion along a track using a DC motor and coordinates with a servo-controlled gate barrier and visual/audio safety indicators.

Objectives:

- **To safely manage car and train interaction at a crossing.**
- **To stop the train automatically at a defined location using an ultrasonic sensor.**
- **To allow both manual and automatic modes of operation using external RC control input.**
- **To stop the train in emergency cases and reset it safely.**
- **To evaluate and compare the system performance using a PID controller.**

Applications:

- **Railway model simulations for education**
- **Mechatronics system integration practice**
- **Low-cost train crossing safety demonstrator**

Overview of System Operation:

- **The train moves on a track powered by a DC motor.**
- **An ultrasonic sensor monitors the train's distance to the crossing.**
- **A PID controller adjusts motor PWM speed to stop the train at a precise point.**
- **A servo motor opens or closes the crossing gate depending on train position or manual input.**
- **A buzzer and LEDs are used for audio/visual signaling.**
- **Emergency and reset pushbuttons allow operator override.**

Main Sensors:

- **Ultrasonic distance sensor (HC-SR04 type)**
- **RC receiver input (FS-iA6B, reading throttle and gate channels)**

Main Actuators:

- **DC motor (train propulsion)**
- **Servo motor (gate)**
- **Buzzer (PA7)**
- **LEDs: Red (PA10) and Green (PA11**

[STM32F103C8T6]
→ PA0: Throttle Input (IC from RC)
→ PA4: Gate Angle Input (IC from RC)
→ PA9: Ultrasonic Trigger
→ PA8: Ultrasonic Echo
→ PA6: PWM Output to DC Motor
→ PA1: PWM to Servo Motor
→ PA7: Buzzer Output
→ PB10, PB11: Mode Selection Buttons (Auto/Manual)
→ PB9: Emergency Stop Button
→ PB7: Reset Button

Part number	Name
HC-SR04	Ultra-Sonic Sensor
3S LIPO Battery 2200 mAH	
Led	LEDS
STM32F103C8T6	STM (BLUE PILL)
5V BUZZER	BUZZER
	BUTTONS
GP2Y0A21YK0F SHARP IR	DISTANCE SENSOR
	DC MOTOR

9G SERVOS	ERVO MOTOR
IDENA 63136	NYLON THREAD
FS-I	

2. Methodology

2.1 Mechanical design

The mechanical system consists of a lightweight toy train driven by a small DC motor connected to the STM32F103C8T6 via a motor driver. The train moves along a fixed track with a sensor-mounting section near the gate area. A servo motor controls a small boom barrier placed across a road model at the crossing point. The barrier pivots between open and closed positions.

Electromechanical integration includes:

- **Secure mounting of ultrasonic sensor facing forward**
- **Servo motor arm attached to gate boom**
- **Motor coupled to the train wheels via pulley or gearbox**
- **Chassis with vibration-damped sensor mount**

2.2 Electrical design

The electrical layout includes:

- **STM32F103C8T6 microcontroller with register-level C code**
- **External 5V supply regulated for sensors and actuators**
- **Motor Driver: L298N or equivalent connected to PA6 PWM and direction pins (PA2, PA3)**
- **Servo Motor connected to PA1 (TIM2_CH2 PWM)**
- **Ultrasonic sensor: Trigger (PA9), Echo (PA8)**
- **RC receiver input signals: CH3 (PA0), CH5 (PA4)**
- **Mode pushbuttons: PB10 (Auto), PB11 (Manual)**
- **Emergency and Reset buttons: PB9 and PB7**
- **Red and Green LEDs on PA10 and PA11**
- **Buzzer on PA7**

Each button input is connected with pull-up resistors and uses debounce timing. The servo is driven by 50 Hz PWM. The motor uses center-aligned PWM at ~1kHz from TIM3. Input capture for RC channels uses TIM2 and TIM4.

2.3 Control

2.3.1. Modeling

The system is modeled as a second-order plant:

Train dynamics approximated by:

$$M \cdot a = F_{motor} - F_{friction}$$
$$F_{motor} = k_{pwm} \cdot PWM_{duty}$$
$$\rightarrow \text{System: } \ddot{x}(t) = k1 \cdot PWM_{duty} - k2 \cdot \dot{x}$$

Laplace domain:

$$X(s)/U(s) = K / (\tau \cdot s + 1)$$

Where:

- X is distance
- U is PWM input
- τ is time constant
- K is gain of motor-plant

2.3.2. Analysis

By assuming no feedback control:

- Output position drifts depending on PWM input
- Without distance feedback, precision stopping is not possible
- Response depends heavily on load and friction

2.3.3. Controller Design

A proportional controller was chosen:

$$u(t) = Kp \cdot (x_{desired} - x_{actual})$$

Where:

- **x_{actual} is from ultrasonic sensor**
- **$x_{desired}$ = 47 cm (target stop point)**

Kp was experimentally tuned (initially 80, later 200) to provide sufficient gain without overshoot.

Closed-loop behavior:

- **Train approaches slowly**
- **As error decreases, motor slows**

- **Stops around target with small settling error**

No integral or derivative terms were used to avoid oscillations and overshoot. Output is clamped to ±8000 PWM max.

2.4 Programming

“Start → Init Peripherals

→ Wait for Mode Input (PB10 = Auto / PB11 = Manual)

↓

Auto Mode:

→ Read Distance from Ultrasonic

→ PID Calculate u

→ Adjust PWM

→ Move Gate (Servo Angle)

→ Check Emergency Button

→ Loop

Manual Mode:

→ Read RC Throttle & Gate

→ Smooth Input (0.7, 0.3 filter)

→ Map Throttle to PWM

→ Move Train

→ Move Gate

→ Check Emergency

→ Loop

Emergency Triggered:

→ Stop Motor

```
→ Turn on Red LED

→ Turn off Green

→ Wait for Reset (PB7)"

#include <stm32f103xb.h>

#include <math.h>

#include "stm32f1xx_hal.h"

#include "stdio.h"

#define RAD_TO_DEG 57.2957795

float actual_distance = 0;

float desired_distance = 47.0f;

float error, u;

float kp = 200;

static float filtered_distance = 0;

volatile uint8_t emergency_stop = 0;

volatile uint32_t throttle_us = 1500;  // From CH3 (PA0)

volatile uint32_t gate_us = 1000;      // From CH5 (PA4)


void debounceDelay() {

   TIM4->PSC = 50 - 1;

   TIM4->ARR = 8000 - 1;

   TIM4->SR = 0;

   TIM4->CR1 = 1;

   while ((TIM4->SR & (1 << 0)) == 0);

   TIM4->CR1 = 0;

}
```

```
uint16_t angle_to_pulse(uint8_t angle) {
    return 1000 + ((angle * 1000) / 180);
}

void set_servo_angle(uint8_t angle) {
    TIM2->CCR2 = angle_to_pulse(angle);
}

void delay(int x) {
    TIM4->PSC = x - 1;
    TIM4->ARR = 8000 - 1;
    TIM4->CR1 |= 1;
    while (!(TIM4->SR & 1));
    TIM4->SR &= ~1;
    TIM4->CR1 &= ~1;
    TIM4->ARR = 0xFFFFFFFF;
    TIM4->PSC = 0;
}

void micro_delay(int x) {
    TIM4->PSC = 8 - 1;
    TIM4->ARR = x - 1;
    TIM4->CR1 |= 1;
    while (!(TIM4->SR & 1));
```

```
    TIM4->SR &= ~1;
    TIM4->CR1 &= ~1;
    TIM4->ARR = 0xFFFFFFFF;
    TIM4->PSC = 0;
}

float Read_Ultrasonic(void) {
    GPIOA->BSRR = (1 << 9);  // Trigger high
    micro_delay(10);
    GPIOA->BRR = (1 << 9);   // Trigger low

    TIM4->CNT = 0;
    TIM4->PSC = 8 - 1;
    TIM4->ARR = 60000 - 1;
    TIM4->CR1 |= TIM_CR1_CEN;

    while (!(GPIOA->IDR & (1 << 8)) && !(TIM4->SR & TIM_SR_UIF));
    if (TIM4->SR & TIM_SR_UIF) return -1;

    uint32_t start = TIM4->CNT;
    while ((GPIOA->IDR & (1 << 8)) && !(TIM4->SR & TIM_SR_UIF));
    uint32_t pulse_width = TIM4->CNT - start;
    TIM4->CR1 &= ~TIM_CR1_CEN;

    return (pulse_width * 0.0343f) / 2;
```

```
}

void TIM4_IRQHandler(void) {
   static uint32_t ch1_rise = 0, ch3_rise = 0;

   // CH1 - PA0 (Throttle)
   if (TIM2->SR & TIM_SR_CC1IF) {
      if (GPIOA->IDR & (1 << 0)) {
         ch1_rise = TIM2->CCR1; // Rising edge time
         TIM2->CCER |= TIM_CCER_CC1P;// Switch to falling edge
      } else {
         throttle_us = TIM2->CCR1 - ch1_rise;// Pulse width = falling - rising
         TIM2->CCER &= ~TIM_CCER_CC1P;  // Reset to rising edge
      }
      TIM2->SR &= ~TIM_SR_CC1IF;
   }

   // CH3 - PA4 (Gate Angle)
   if (TIM4->SR & TIM_SR_CC3IF) {
         if (GPIOB->IDR & (1 << 8)) {
            ch3_rise = TIM4->CCR3;
            TIM4->CCER |= TIM_CCER_CC3P;
         } else {
            gate_us = TIM4->CCR3 - ch3_rise;
            TIM4->CCER &= ~TIM_CCER_CC3P;
```

```
        }
        TIM4->SR &= ~TIM_SR_CC3IF;
    }

}

uint16_t map_throttle_to_pwm(uint32_t us) {
  if (us < 1000) us = 1000;
  if (us > 2000) us = 2000;
  return (us - 1000) * 8;
}

void ms_delay(uint32_t ms) {
  for (volatile uint32_t i = 0; i < ms * 8000; i++) __asm("nop");
}

int main(void) {
   RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPBEN |
RCC_APB2ENR_AFIOEN;
  RCC->APB1ENR |= RCC_APB1ENR_TIM2EN | RCC_APB1ENR_TIM3EN |
RCC_APB1ENR_TIM4EN;

  GPIOA->CRL = 0x2A4822A4;  // PA0 (input), PA1 (servo), PA4 (input)
  GPIOA->CRH = 0x44442228;  // PA7 (buzzer), PA8 (echo), PA9 (trig)
  GPIOB->CRL = 0x84444444;
  GPIOB->CRH = 0x44448884;
```

```
GPIOB->ODR |= (1<<9)|(1 << 10) | (1 << 11) | (1<<7) | (1<<8);


GPIOA->ODR |= (1 << 2);   // motor forward
GPIOA->ODR &= ~(1 << 3);


// PWM - DC Motor (PA6 TIM3_CH1)
TIM3->ARR = 8000;
TIM3->CCER |= (1 << 0);
TIM3->CCMR1 = (1 << 3) | (6 << 4);
TIM3->CR1 = (1 << 0) | (2 << 5) | (1 << 7);
TIM3->EGR = 1;


// Servo - PA1 (TIM2_CH2)
TIM2->PSC = 8 - 1;
TIM2->ARR = 20000 - 1;
TIM2->CCR2 = 1500;
TIM2->CCMR1 |= (6 << 12) | (1 << 11);
TIM2->CCER |= (1 << 4);
TIM2->CR1 |= 1;
TIM2->EGR = 1;


// --- Input Capture for manual mode ---
GPIOA->CRL &= ~((0xF << 0) | (0xF << 16));
GPIOA->CRL |=  ((0x4 << 0) | (0x4 << 16)); // PA0, PA4 input floating
```

```
    TIM2->CCMR1 |= (1 << 0);  // CC1S = 01 (PA0)
    TIM2->CCER |= (1 << 0);   // Enable CH1
    TIM2->DIER |= (1 << 1);   // CC1 interrupt

    TIM4->CCMR2 &= ~(0xFF);
    TIM4->CCMR2 |= (1 << 0);   // CC3S = 01
    TIM4->CCER |= (1 << 8);    // CC3E
    TIM4->DIER |= (1 << 3);    // CC3 interrupt enable
    TIM4->CR1 |= 1;            // Start TIM4


    NVIC_EnableIRQ(TIM4_IRQn);
  //  TIM2->CR1 |= 1;  // Start timer


    while (1) {
       if ((GPIOB->IDR & (1 << 10)) == 0) { // AUTOMATIC MODE
          debounceDelay();
          if ((GPIOB->IDR & (1 << 10)) == 0) {
             while (!emergency_stop) {
//                   uint32_t sum = 0;
//                   for (int i = 0; i < 2; i++) {
//                   sum +=Read_Ultrasonic() ;}
//                  actual_distance = sum / 3;
                  actual_distance = Read_Ultrasonic();
```

```
    filtered_distance = 0.7f * filtered_distance + 0.3f * actual_distance;
    if (filtered_distance < 4) filtered_distance = 4;
    if (filtered_distance > 60) filtered_distance = 60;
//   if (filtered_distance > 39) filtered_distance +=1 ;

    error = desired_distance - filtered_distance;
    u = error * kp;
    if (u > 8000) u = 8000;
    if (u < -8000) u = -8000;

    if (u > 0) {
       GPIOA->ODR |= (1 << 2);
       GPIOA->ODR &= ~(1 << 3);
       TIM3->CCR1 = u;
    } else if (u < 0) {
       GPIOA->ODR &= ~(1 << 2);
       GPIOA->ODR |= (1 << 3);
       TIM3->CCR1 = -u;
    } else {
       TIM3->CCR1 = 0;
       GPIOA->ODR &= ~(1 << 2);
       GPIOA->ODR &= ~(1 << 3);
    }

    if (filtered_distance > 40) {
```

```
            set_servo_angle(0);
            GPIOA->ODR &= ~(1 << 7);
        } else if (filtered_distance > 17 && filtered_distance <= 40) {
            set_servo_angle(200);
            GPIOA->ODR |= (1 << 7);
        } else {
            set_servo_angle(0);
            GPIOA->ODR ^= (1 << 7);
            micro_delay(500);
        }
        // Check emergency stop ES button
            if ((GPIOB->IDR & (1 << 9)) == 0) {
                    debounceDelay();
                    if ((GPIOB->IDR & (1 << 9)) == 0) {
                                debounceDelay();
                            emergency_stop = 1;
                            GPIOA->ODR|= (1<<10); // red light
                            GPIOA->ODR &= ~(1 << 11); // turn off
green light
                            // Stop motor and buzzer immediately
                            TIM3->CCR1 = 0;
                            GPIOA->ODR &= ~((1 << 2) | (1 << 3));  //
Stop motor
                            GPIOA->ODR &= ~(1 << 7);            //
Buzzer off
                            break;  // Exit mode
```

```
                                        }
                    }


                micro_delay(100);
              }
            }
          }
          // check reset button
          if ((GPIOB->IDR & (1 << 7)) == 0) {
           debounceDelay();
           if ((GPIOB->IDR & (1 << 7)) == 0) {
            emergency_stop = 0;
            GPIOA->ODR|= (1<<11); //  green led
            GPIOA->ODR &= ~(1 << 10); // turn off red light
           }
          }


          if ((GPIOB->IDR & (1 << 11)) == 0) {  // MANUAL MODE
             debounceDelay();
             if ((GPIOB->IDR & (1 << 11)) == 0) {
                    while (!emergency_stop) {
                       // --- Read throttle & gate pulse widths ---
                       actual_distance = Read_Ultrasonic();
                       if (actual_distance < 2 || actual_distance > 80 || actual_distance < 0)
continue;
```

```
            filtered_distance = 0.7f * filtered_distance + 0.3f * actual_distance;

            // --- Apply throttle from PA0 ---
            u = map_throttle_to_pwm(throttle_us);  // Converts 1000-2000us to
0-8000
            if (u > 0) {
               GPIOA->ODR |= (1 << 2);
               GPIOA->ODR &= ~(1 << 3);
               TIM3->CCR1 = u;
            } else {
               TIM3->CCR1 = 0;
               GPIOA->ODR &= ~(1 << 2);
               GPIOA->ODR &= ~(1 << 3);
            }

            // --- Gate angle from PB8 (e.g., CH5 VRA knob) ---
            if (gate_us < 1000) gate_us = 1000;
            if (gate_us > 2000) gate_us = 2000;
            uint8_t gate_angle = (gate_us - 1000) * 180 / 1000;
            set_servo_angle(gate_angle);  // Sets TIM2->CCR2

            // --- Buzzer logic based on gate angle only ---
            if (gate_angle >= 160) {
               GPIOA->ODR |= (1 << 7);  // Buzzer ON when gate is closed
            } else if (gate_angle <= 20) {
```

```
            GPIOA->ODR &= ~(1 << 7); // Buzzer OFF when gate is open
        }
        // Check emergency stop
        if ((GPIOB->IDR & (1 << 9)) == 0) {
            emergency_stop = 1;
            GPIOA->ODR|= (1<<10); // red light
              GPIOA->ODR &= ~(1 << 11); // turn off green light
            // Stop everything
            TIM3->CCR1 = 0;
            GPIOA->ODR &= ~((1 << 2) | (1 << 3));  // Motor off
            GPIOA->ODR &= ~(1 << 7);              // Buzzer off
            break;  // Exit manual mode
          }

        ms_delay(50);  // Adjust delay as needed
      }



  }
}
if ((GPIOB->IDR & (1 << 7)) == 0) {
    debounceDelay();
    if ((GPIOB->IDR & (1 << 7)) == 0) {
     emergency_stop = 0;
     GPIOA->ODR|= (1<<11); //  green led
```

```
            GPIOA->ODR &= ~(1 << 10); // turn off red light
          }
        }
    }
}
```

3. Design Evaluation

The system successfully switches between manual and automatic modes. In automatic mode, the PID control enables smooth stopping of the train at ~47 cm. Gate logic synchronizes well, closing when the train approaches and opening when it clears the zone. Emergency and reset buttons override the system correctly.

Observed errors:

- **Minor instability in RC signal due to timing glitches.**
- **Servo fluttering resolved by filtering gate_us signal.**
- **Ultrasonic sensor occasionally reads out-of-range; capped via clamping.**
- **Emergency mode required careful debouncing.**

Results vs Simulation:

- **Control law matches expected behavior.**
- **Real-world friction and lag introduced minor delay.**
- **Filtered distance smoothing improved accuracy and gate reliability.**

4. Appendix

PA0: Input Capture from RC CH3 (Throttle)

- **PA4: Input Capture from RC CH5 (Gate Angle)**
- **PA6: TIM3_CH1 → PWM to DC Motor Driver**
- **PA1: TIM2_CH2 → PWM to Servo Motor**
- **PA9: Trigger (Output) to Ultrasonic Sensor**
- **PA8: Echo (Input) from Ultrasonic Sensor**
- **PB10: Auto Mode Button**
- **PB11: Manual Mode Button**
- **PB9: Emergency Stop Button**
- **PB7: Reset Button**
- **PA7: Buzzer (Digital output)**
- **PA10: Red LED**
- **PA11: Green LED**

Timers Used:

- **TIM2: Servo PWM and Input Capture (Throttle)**
- **TIM3: Motor PWM**
- **TIM4: Delay Generation, Input Capture (Gate), Ultrasonic Echo Timing**